interfaces may have the same driver—though, of course, one interface can have a maximum of one driver only.

It is okay and fairly common to have a single USB device driver for all the interfaces of a USB device. The *"Driver=..."* entry in the *proc* window output (Figure 3) shows the interface to driver mapping—a "(none)" indicating no associated driver. For every interface, there would be one or more end-points. An end-point is like a pipe for transferring information either into or from the interface of the device, depending on the functionality. Based on the type of information, the endpoints have four types: *Control*, *Interrupt*, *Bulk* and *Isochronous.*

As per the USB protocol specification, all valid USB devices have an implicit special control end-point zero, the only bi-directional end-point. Figure 4 shows the complete pictorial representation of a valid USB device, based on the above explanation.

Coming back to the USB device sections (Figure 3), the first letter on each line represents the various parts of the USB device specification just explained. For example, D for device, C for configuration, I for interface, E for endpoint, etc. Details about these and various others are available in the kernel source, in *Documentation/usb/proc_usb_info.txt.*

The USB pen drive driver registration

"Seems like there are so many things to know about the USB protocol, to be able to write the first USB driver itself—device configuration, interfaces, transfer pipes, their four types, and so many other symbols like T, B, S, … under a USB device specification," sighed Shweta. "Yes, but don't you worry—all of that can be covered in detail later. Let's do first things first—get the pen drive's interface associated with our USB device driver *(pen_register.ko),*" consoled Pugs. Like any other Linux device driver, here, too, the constructor and the destructor are required—basically the same driver template that has been used for all the drivers. However, the content would vary, as this is a hardware protocol layer driver, i.e., a horizontal driver, unlike a character driver, which was one of the vertical drivers discussed earlier. The difference would be that instead of registering with and unregistering from VFS, here this would be done with the corresponding protocol layer—the USB core in this case; instead of providing a user-space interface like a device file, it would get connected with the actual device in hardware-space. The USB core APIs for the same are as follows (prototyped in <*linux/usb.h*>):

```
int usb_register(struct usb_driver *driver);
void usb_deregister(struct usb_driver *);
```

As part of the *usb_driver* structure, the fields to be provided are the driver's name, ID table for auto-detecting the particular device, and the two callback functions to be invoked by the USB core during a hot plugging and a hot removal of the device, respectively. Putting it all together, *pen_register.c* would look like what follows:

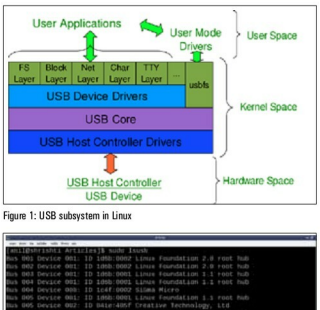


Figure 1: USB subsystem in Linux



Figure 2: Output of *lsusb*

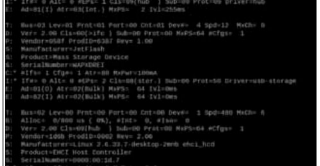


Figure 3: USB's proc window snippet

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/usb.h>


static int pen_probe(struct usb_interface *interface, const
struct usb_device_id *id)
{
    printk(KERN_INFO "Pen drive (%04X:%04X) plugged\n", id-
>idVendor, id->idProduct);
```